

Python ML/AI Training Document

Version: 1.0

Python ML/AI Training Document

Python History

Python was conceived in the late 1980s by Guido van Rossum at Centrum Wiskunde & Informatica (CWI) in the Netherlands as a successor to ABC programming language, which was inspired by SETL, capable of exception handling and interfacing with the Amoeba operating system. Its implementation began in December 1989.

1. Why the Name Python?
2. Usage of Python
3. Resources
 - a. [Python History and Versions](#)
 - b. [History of Python](#)

Why Python?

The python language is one of the most accessible programming languages available because it has simplified syntax and is not complicated, which gives more emphasis on natural language. Due to its ease of learning and usage, python codes can be easily written and executed much faster than other programming languages.

1. Features
2. Resources
 - a. [Python Features](#)
 - b. [Top 10 Reasons Why Python is So Popular With Developers in 2021](#)
 - c. [Python 2 vs Python 3: Which One You Should Learn](#)

Syntax

Python was designed to be a highly readable language. It has a relatively uncluttered visual layout and uses English keywords frequently where other languages use punctuation. Python aims to be simple and consistent in the design of its syntax.

1. Difference between Compiled and Interpreted Language
2. Loosely Typed vs Strongly Typed Languages
3. Indentation
4. Variables
5. Comments
6. Resources
 - a. [Difference between Compiled and Interpreted Language](#)
 - b. [Loosely typed vs strongly typed languages](#)
 - c. [Execute Python Syntax](#)
 - d. [How to Use Python: Your First Steps – Real Python In this step-by-step tutorial, you](#)
 - e. [Learn Python Programming - Python Tutorial](#)

Variables

Python supports different types of variables (data types) such as whole numbers, floating point numbers and text.

You do not need to specify the datatype of a variable, you can simply assign any value to a variable.

1. Dynamic Typing vs Static Typing
2. Resources
 - a. [Static vs. Dynamic Typing](#)
 - b. [Variables and Types](#)
 - c. [W3schools Python Variables](#)
 - d. [Python Tutorial Python Variables, Constants and Literals](#)

Operators

Operators are special symbols in Python that carry out arithmetic or logical computation. The value that the operator operates on is called the operand.

1. Arithmetic operators
2. Comparison operators
3. Logical operators
4. Bitwise operators
5. Assignment operators
6. Specials operators
 - a. Identity operators
 - b. Membership operators
7. Resources
 - a. [W3Schools Python Operators](#)
 - b. [Python Operators: Arithmetic, Comparison, Logical and more.](#)

What is Pip

pip is the default package installer for Python. It is used to install, upgrade, and manage Python packages and their dependencies.

Here are some key points to keep in mind when working with pip:

1. pip is included with Python versions 3.4 and later, so you don't need to install it separately.
2. You can use pip to install packages from the Python Package Index (PyPI) or from other sources like version control repositories.
3. You can use pip to install packages globally or in a virtual environment.
4. You can use pip to manage package dependencies by creating and installing from a requirements file.

Here are some examples of how to use pip:

1. To install a package: `pip install package_name`
2. To upgrade a package: `pip install --upgrade package_name`
3. To uninstall a package: `pip uninstall package_name`
4. To list installed packages: `pip list`
5. To create a requirements file: `pip freeze > requirements.txt`
6. To install packages from a requirements file: `pip install -r requirements.txt`

It's important to keep your packages up-to-date and to manage their dependencies carefully to ensure that your code runs smoothly.

Resources:

- <https://realpython.com/what-is-pip/>

Control Structures

Control Structures are the blocks that analyze variables and choose directions in which to go based on given parameters. The basic Control Structures in programming languages are: Conditionals (or Selection): which are used to execute one or more statements if a condition is met.

1. Iteration (for, while)
2. Selection (if, else)
3. Resources
 - a. [1.10. Control Structures — Problem Solving with Algorithms and Data Structures](#)
 - b. [loops in python](#)

Type Casting

Typecasting, or type conversion, is a method of changing an entity from one data type to another. It is used in computer programming to ensure variables are correctly processed by a function. An example of typecasting is converting an integer to a string.

1. Implicit vs Explicit Type Casting
2. Resources
 - a. [Python Type Conversion and Type Casting \(With Examples\)](#)
 - b. [Python Casting Tutorial](#)

Exception Handling

Python has many built-in exceptions that are raised when your program encounters an error (something in the program goes wrong).

When these exceptions occur, the Python interpreter stops the current process and passes it to the calling process until it is handled. If not handled, the program will crash.

1. try-except
2. try-except-else

3. try-except-else-finally
4. Resources
 - a. [Python Tutorial Python Exception Handling Using try, except and finally statement](#)
 - b. [Python Try Except – w3schools.com](#)

Functions

In Python, a function is a group of related statements that performs a specific task. Functions help break our program into smaller and modular chunks. As our program grows larger and larger, functions make it more organized and manageable. Furthermore, it avoids repetition and makes the code reusable.

1. Syntax
2. Partial functions
3. Anonymous/lambda functions
4. Recursive functions
5. Monkey patching
6. Resources
 - a. [Python Functions \(def\): Definition with Examples](#)
 - b. [Partial Functions in Python](#)
 - c. [Python Lambda \(Anonymous\) Function](#)
 - d. [Thinking Recursively in Python – Real Python](#)
 - e. [Monkey Patching in Python \(Dynamic Behavior\)](#)

Collections (Arrays)

There are four collection data types in the Python programming language:

1. List is a collection which is ordered and changeable. Allows duplicate members.
 - a. Slicing of a List
 - b. List Comprehension
 - c. Operations on List
 - d. List Methods
 - e. Resources
 - i. [Python Lists](#)
 - ii. [Python List \(With Examples\)](#)
 - iii. [Python - Lists](#)
2. Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
 - a. Converting list to a Tuple
 - b. Concatenation of Tuples
 - c. Nesting of Tuples
 - d. Slicing in Tuples
 - e. Resources
 - i. [Detailed article on Tuples in Python](#)
 - ii. [Python Tuple \(With Examples\)](#)

- iii. [Python - Tuples](#)
- 3. Set is a collection which is unordered and unindexed. No duplicate members.
 - a. Frozen sets
 - b. Union, Intersection and Difference of Sets
 - c. Resources
 - i. [Sets in Python](#)
 - ii. [Python Set \(With Examples\)](#)
 - iii. [Python - Sets](#)
- 4. Dictionary is a collection which is ordered (prior to Python 3.6, dictionaries were unordered) and changeable. No duplicate members.
 - a. Nested Dictionary
 - b. pop() vs popItem()
 - c. Resources
 - i. [Recent Articles on Python Dictionary](#)
 - ii. [Python Dictionary \(With Examples\)](#)
 - iii. [Python - Dictionary](#)

Strings

In Python, Strings are arrays of bytes representing 1 character. However, Python does not have a character data type, a single character is simply a string with a length of 1. Square brackets can be used to access elements of the string.

- 1. Escape Sequencing in Python
- 2. Formatting of Strings
- 3. String Methods
- 4. Resources
 - a. [Recent Articles on Python String](#)
 - b. [Python Strings \(With Examples\)](#)
 - c. [Python - Strings](#)

Object Oriented Programming (OOP)

Object-Oriented Programming(OOP), is all about creating “objects”. An object is a group of interrelated variables and functions. These variables are often referred to as properties of the object and functions are referred to as the behavior of the objects. These objects provide a better and clear structure for the program.

Classes and Objects

A class is a collection of objects. Unlike the primitive data structures, classes are data structures that the user defines. They make the code more manageable.

When we define a class only the description or a blueprint of the object is created. There is no memory allocation until we create its object. The object instance contains real data or information.

- 1. The Life Cycle of Python Instance Objects

2. Resources

- a. [Object Oriented Programming in Python | OOPs Concepts Python](#)
- b. [Python - Object Oriented](#)
- c. [The Life Cycle of Python Instance Objects | by Yong Cui](#)

Constructors and Class Methods

Constructors are generally used for instantiating an object. The task of constructors is to initialize(assign values) to the data members of the class when an object of class is created. In Python the `__init__()` method is called the constructor and is always called when an object is created.

A class method is a method which is bound to the class and not the object of the class. They have access to the state of the class as it takes a class parameter that points to the class and not the object instance. It can modify a class state that would apply across all the instances of the class.

1. Types of constructors
2. Resources
 - a. [classmethod\(\) in Python](#)
 - b. [Constructors in Python](#)
 - c. [Python's Instance, Class, and Static Methods Demystified – Real Python](#)

Inheritance and its types

Inheritance is the capability of one class to derive or inherit the properties from another class.

1. Single inheritance
2. Multi-level inheritance
3. Multiple inheritance
4. Hierarchical inheritance
5. Hybrid inheritance
6. Resources
 - a. [Types of Inheritance in Python | Python Inheritance \[With Example\]](#)
 - b. [Inheritance in Python](#)

Polymorphism

The literal meaning of polymorphism is the condition of occurrence in different forms.

Polymorphism is a very important concept in programming. It refers to the use of a single type entity (method, operator or object) to represent different types in different scenarios.

1. Run-time
2. Compile-time
3. Resources
 - a. [Polymorphism in Python\(with Examples\)](#)
 - b. [Polymorphism in Python | Object Oriented Programming \(OOPs\)](#)

Static Variables and Static Methods

Class or Static variables are the variables that belong to the class and not to objects. Class or Static variables are shared amongst objects of the class. All variables which are assigned a value in the class declaration are class variables. And variables which are assigned values inside class methods are instance variables.

1. Class Method vs Static Method
2. Resources
 - a. [Python Static Variables and Methods](#)
 - b. [class method vs static method in Python](#)

Function Overloading and Function Overriding

Function overloading is a feature of object oriented programming where two or more functions can have the same name but different parameters.

Function overriding is a feature that allows us to have the same function in the child class which is already present in the parent class. A child class inherits the data members and member functions of parent class, but when you want to override a functionality in the child class then you can use function overriding.

1. Advantages and Disadvantages of function overloading
2. Advantages and Disadvantages of function overriding
3. Resources
 - a. [Difference between Method Overloading and Method Overriding in Python](#)
 - b. [Python Inheritance Tutorial- Method Overloading & Method Overriding](#)

Advance Topics

File Handling

Python treats files differently as text or binary and this is important. Each line of code includes a sequence of characters and they form a text file. Each line of a file is terminated with a special character, called the EOL or End of Line characters like comma {,} or newline character. It ends the current line and tells the interpreter a new one has begun.

1. Working of open() function
2. Using write along with with() function
3. Resources
 - a. [Basics of File Handling in Python](#)
 - b. [Python File I/O: Read and Write Files in Python](#)
 - c. [Python - Files I/O](#)

Iterators

Iterator in python is an object that is used to iterate over iterable objects like lists, tuples, dicts, and sets. The iterator object is initialized using the iter() method. It uses the next() method for iteration.

1. Building Custom Iterators
2. Python Infinite Iterators
3. Resources
 - a. [Python Iterators \(`iter` and `next` \): How to Use it and Why?](#)
 - b. [Python Iterators](#)

Generators

A generator is a function that returns an object (iterator) which we can iterate over (one value at a time).

1. Differences between Generator function and Normal function
2. Python Generators with a Loop
3. Pipelining Generators
4. Reading Large File
5. Resources
 - a. [Python yield, Generators and Generator Expressions](#)
 - b. [Generators in Python](#)
 - c. [How to Use Generators and yield in Python – Real Python](#)

Decorators

A decorator is a design pattern in Python that allows a user to add new functionality to an existing object without modifying its structure. Decorators are usually called before the definition of a function you want to decorate.

1. First Class Objects
2. Chaining Decorators
3. Additional Resources:
 - a. [Decorators in Python](#)
 - b. [Python Decorators: How to Use it and Why?](#)
 - c. [Primer on Python Decorators](#)

Shallow Copy vs Deep Copy

A shallow copy constructs a new compound object and then (to the extent possible) inserts references into it to the objects found in the original. A deep copy constructs a new compound object and then, recursively, inserts copies into it of the objects found in the original.

1. Resources
 - a. [copy in Python \(Deep Copy and Shallow Copy\)](#)
 - b. [copy — Shallow and deep copy operations — Python 3.9.5 documentation](#)

Random Numbers

1. Pseudorandom Number Generators
2. Random Numbers with the Python Standard Library
3. Resources
 - a. [How to Generate Random Numbers in Python](#)

- b. [Generating random number list in Python - tutorialspoint.com](https://www.tutorialspoint.com/python/python_random_numbers.htm)

Numpy

NumPy is the fundamental package for scientific computing in Python. NumPy arrays facilitate advanced mathematical and other types of operations on large numbers of data.

1. Numpy arrays and functions
2. Resources
 - a. [Introduction to NumPy](https://www.numpy.org/en/stable/)
 - b. [NumPy quickstart — NumPy v1.21 Manual](https://numpy.org/doc/1.21/)
 - c. [Python Numpy Tutorial | Learn Numpy Arrays With Examples](https://www.datacamp.com/courses/python-numpy-tutorial/)

Pandas

Pandas is a Python library used for working with data sets. It has functions for analyzing, cleaning, exploring, and manipulating data.

RESOURCES:

- <https://www.geeksforgeeks.org/introduction-to-pandas-in-python/>
- https://www.w3schools.com/python/pandas/pandas_getting_started.asp

Prerequisites

Python style guide, Linter and Formatter

Python style guide (PEP 8) gives coding conventions for the python code comprising the standard library in the main Python distribution. Linting highlights syntactical and stylistic problems in your Python source code, which oftentimes helps you identify and correct subtle programming errors or unconventional coding practices that can lead to errors. For example, linting detects use of an uninitialized or undefined variable, calls to undefined functions, missing parentheses, and even more subtle issues such as attempting to redefine built-in types or functions.

- a. Python style guide (PEP 8)
- b. What is Linter ?
- c. What is a formatter ?
- d. Difference Between Linter and Formatter?
- e. Resources:
 - i. [PEP 8 -- Style Guide for Python Code](https://www.python.org/dev/peps/pep-0008/)
 - ii. [Linting Python](https://www.linting.com/)
 - iii. [Python Code Quality: Tools & Best Practices – Real Python](https://realpython.com/python-code-quality/)

SOPs

1. Using all you have learnt about the language(basics), **build some example programs(suggestions below)**, workout with your buddy to decide which programs to implement.
Preferably something which includes dealing with files + some kind of parsing(CSVs) + some data generation + some network calls.
2. Get these **codes reviewed** by your buddy. And these example codes should not be discarded, and might be reviewed by the training department.
3. **Once you are done with practice problems, upload them in a repo on github and send the repo link with your next daily standup(both in email and on slack).**
 - a. Don't commit in master, make a branch named dev and add all commits to it, and make a PR, so we can add review comments and keep track of the comments in there.
 - b. *Repo should be private with only your **buddy and training department** (ziaulrehman40) in it.*
4. *Get the code you write for each problem reviewed by your buddy properly, fix all feedback received before moving to the next problem and before evaluation by the training department. Code should be modular, properly split in modules, classes/files etc.*

Multiprocessing in Python

Multiprocessing is a way to utilize multiple CPU cores in order to perform tasks simultaneously. This can greatly increase the speed of certain operations in Python.

To use multiprocessing in Python, you'll need to follow these steps:

1. Import the multiprocessing module.
2. Define the function that you want to run in parallel. This function should take a single argument, which will be the input for that process.
3. Create a multiprocessing pool object.
4. Use the pool object to apply your function to multiple inputs. The map() function will apply your function to each item in the input list and return a list of outputs.
5. When you're done with the pool, close it.

It's important to note that multiprocessing is useful for CPU-bound tasks, such as numerical computations. However, it can be less effective for I/O-bound tasks, such as reading and writing files or making network requests.

RESOURCES:

- <https://docs.python.org/3/library/multiprocessing.html>

- <https://www.geeksforgeeks.org/multiprocessing-python-set-1/>

Multithreading in Python

Multithreading is a way to perform multiple tasks simultaneously within a single process. Unlike multiprocessing, which utilizes multiple CPU cores, multithreading uses a single CPU core to switch between tasks rapidly. Here are some key points to keep in mind when working with multithreading in Python:

1. Multithreading is useful for I/O-bound tasks, such as reading and writing files or making network requests.
2. Python's Global Interpreter Lock (GIL) prevents multiple threads from executing Python bytecode simultaneously. This means that multithreading in Python is not suitable for CPU-bound tasks, such as numerical computations.
3. To create a new thread in Python, you can use the threading module.

RESOURCES:

- <https://docs.python.org/3/library/threading.html>
- <https://www.geeksforgeeks.org/multithreading-python-set-1/>
- https://www.tutorialspoint.com/python/python_multithreading.htm

Async/Await in Python

Async/await is a way to write asynchronous code in Python. This allows you to perform I/O-bound tasks without blocking the main thread of execution. Here are some key points to keep in mind when working with async/await in Python:

1. Async/await is useful for I/O-bound tasks, such as reading and writing files or making network requests.
2. Async/await uses coroutines to allow for non-blocking I/O.
3. To define an async function in Python, you use the `async def` syntax.
4. To call an async function, you use the `await` keyword.
5. It's important to note that async/await is not suitable for CPU-bound tasks, such as numerical computations.

RESOURCES:

- <https://docs.python.org/3/library/asyncio.html>
- <https://docs.python.org/3/library/asyncio-task.html>

- <https://realpython.com/async-io-python/>

Abstract Classes in Python

Abstract classes are classes that cannot be instantiated on their own. Instead, they are meant to be subclassed and provide a blueprint for other classes to follow. Here are some key points to keep in mind when working with abstract classes in Python:

1. Abstract classes are useful for defining common interfaces that other classes can implement.
2. Abstract classes cannot be instantiated on their own. Instead, you must create a subclass that implements all of the abstract methods.
3. To define an abstract class in Python, you use the `abc.ABC` class.
4. To define an abstract method in Python, you use the `@abc.abstractmethod` decorator.

It's important to note that abstract classes are not meant to be used directly, but rather as a way to define common interfaces for other classes to follow.

RESOURCES:

- <https://docs.python.org/3/library/abc.html#abc.abstractmethod>
- <https://www.geeksforgeeks.org/abstract-classes-in-python/>

Dependency Injection in Python

Dependency injection is a design pattern that allows you to write more modular and testable code. The basic idea behind dependency injection is that instead of creating objects directly, you pass them as arguments to the constructor or method of another object. Here are some key points to keep in mind when working with dependency injection in Python:

1. Dependency injection is useful for creating more modular and testable code.
2. Instead of creating objects directly, you pass them as arguments to the constructor or method of another object.
3. Dependency injection can be done manually or using a framework.
4. In Python, the `__init__()` method is used to inject dependencies into a class.
5. It's important to note that while dependency injection can lead to more modular and testable code, it can also make the code more complex and difficult to understand.

RESOURCES:

- https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html

- <https://www.netguru.com/blog/dependency-injection-with-python-make-it-easy>

Factory Pattern in Python

The factory pattern is a way to create objects without specifying the exact class of object that will be created. Instead, you use a factory method that returns an instance of the appropriate class based on the input parameters. Here are some key points to keep in mind when working with the factory pattern in Python:

1. The factory pattern is useful for creating objects when you don't know exactly what type of object you need at runtime.
2. The factory method returns an instance of the appropriate class based on the input parameters.
3. This makes it easy to add new types of objects without changing the code that uses them.
4. There are several ways to implement the factory pattern in Python, such as using a simple function, a class method, or a separate factory class.

It's important to note that while the factory pattern can make your code more flexible and extensible, it can also make it more complex and harder to understand. Use it judiciously and only when it makes sense for your particular use case.

RESOURCES:

- <https://realpython.com/factory-method-python/>
- <https://www.geeksforgeeks.org/factory-method-python-design-patterns/>

List and Dictionary Comprehension in Python

List and dictionary comprehension is a way to create new lists and dictionaries in a concise and Pythonic way. It allows you to create a new list or dictionary by applying an expression to each item in an existing list or dictionary. Here are some key points to keep in mind when working with list and dictionary comprehension in Python:

1. List comprehension is useful for creating new lists based on existing lists.
2. Dictionary comprehension is useful for creating new dictionaries based on existing dictionaries.
3. List and dictionary comprehension can be used to filter and transform items in the original list or dictionary.
4. List and dictionary comprehension can be more concise and easier to read than using traditional loops.

It's important to note that while list and dictionary comprehension can make your code more concise and readable, it can also make it harder to understand for beginners. Use it judiciously and only when it makes sense for your particular use case.

RESOURCES:

- <https://realpython.com/list-comprehension-python/>
- <https://realpython.com/python-dicts/#dictionary-comprehension>

Pyenv, Virtualenv, and Poetry in Python

Pyenv, virtualenv, and Poetry are tools used in Python for managing different versions of Python and creating isolated environments for your projects. Here are some key points to keep in mind when working with these tools:

1. Pyenv is a tool for managing multiple versions of Python on your system. It allows you to easily switch between different versions of Python.
2. Virtualenv is a tool for creating isolated Python environments. It allows you to install packages specific to a particular project without affecting other projects on your system.
3. Poetry is a tool for managing dependencies in your Python projects. It allows you to easily install and manage packages, as well as create virtual environments for your projects.

It's important to note that while these tools can make your life easier when working with Python, they can also add complexity to your workflow. Use them judiciously and only when they make sense for your particular use case.

RESOURCES:

- <https://github.com/pyenv/pyenv>
- <https://virtualenv.pypa.io/en/latest/>
- <https://python-poetry.org/docs/>

VMs, Docker, and Containers

VMs (Virtual Machines), Docker, and containers are all technologies used for managing software applications. Here are some key points to keep in mind when working with these technologies:

1. VMs are a way of running multiple operating systems on a single physical machine. Each VM runs its own operating system and has its own set of resources, such as CPU, memory, and storage.
2. Docker is a containerization platform that allows you to package an application and its dependencies into a container. A container is a lightweight, standalone executable package that includes everything needed to run the application.
3. Containers are a way of packaging software so that it can run consistently across different computing environments. Containers are isolated from the host system and from other containers, which makes them more secure and easier to manage.

It's important to note that while VMs, Docker, and containers all have their own strengths and weaknesses, they are all useful tools for managing software applications. VMs are often used when you need to run multiple operating systems on a single physical machine, while Docker and containers are often used when you need to package an application and its dependencies into a portable and consistent format.\

RESOURCES:

- <https://www.vmware.com/topics/glossary/content/virtual-machine.html>
- <https://www.docker.com/resources/what-container/>

Garbage Collection in Python

Garbage collection is the process of automatically freeing up memory that is no longer being used by a program. In Python, garbage collection is handled by a mechanism called reference counting. Here are some key points to keep in mind when working with garbage collection in Python:

1. Reference counting is a technique where each object keeps track of how many references to it exist in the program.
2. When an object's reference count drops to zero, it is no longer being used and can be safely deleted.
3. Python also has a secondary garbage collector that runs periodically to clean up any objects that have circular references.
4. Circular references occur when two or more objects reference each other, creating a cycle that cannot be broken by reference counting alone.

It's important to note that while Python's garbage collector is generally very effective, it can still be possible to create memory leaks in your code if you're not careful. It's a good idea to regularly monitor your program's memory usage and use tools like `memory_profiler` to identify any potential issues.

RESOURCES:

- <https://towardsdatascience.com/memory-management-and-garbage-collection-in-python-c1cb51d1612c>
- <https://www.geeksforgeeks.org/garbage-collection-python/>

The yield Keyword in Python

The yield keyword in Python is used to define a generator function. A generator function is a special type of function that returns an iterator, which can be used to iterate over a sequence of values. Here are some key points to keep in mind when working with the yield keyword in Python:

1. A generator function is defined using the def keyword, but instead of returning a value using the return keyword, it uses the yield keyword.
2. When a generator function is called, it returns an iterator object, which can be used to iterate over the values generated by the function.
3. Each time the yield keyword is encountered in the generator function, it returns the current value and saves the state of the function so that it can be resumed later.
4. Generator functions are useful for generating large sequences of values that can't be generated all at once.

It's important to note that while generators can be very useful for generating large sequences of values, they can also be more complex to work with than regular functions. It's also important to understand how generators work under the hood so that you can avoid common pitfalls like accidentally consuming the iterator multiple times.

RESOURCES:

- <https://www.geeksforgeeks.org/python-yield-keyword/>
- <https://www.geeksforgeeks.org/use-yield-keyword-instead-return-keyword-python/>
- <https://www.simplilearn.com/tutorials/python-tutorial/yield-in-python>

Context Managers in Python

Context managers in Python are a way to manage resources that need to be acquired and released properly, such as files, network connections, and locks. They allow you to define a block of code that will be executed with a resource and ensure that the resource is properly released when the block is exited, even if an exception is raised. Here are some key points to keep in mind when working with context managers in Python:

1. A context manager is defined using the with statement, which creates a new context for the code block.
2. The with statement takes an expression that returns a context manager object.
3. The context manager object must define two methods: `__enter__()` and `__exit__()`.
4. The `__enter__()` method is called when the block is entered, and it returns the resource that will be managed by the context manager.
5. The `__exit__()` method is called when the block is exited, and it releases the resource.

Context managers are useful for managing resources that need to be acquired and released properly, such as files, network connections, and locks. They ensure that resources are properly released even if an error occurs.

It's important to note that while context managers can make your code more robust and reliable, they can also add complexity to your code. Use them judiciously and only when they make sense for your particular use case.

RESOURCES:

- <https://realpython.com/python-with-statement/>
- <https://www.geeksforgeeks.org/context-manager-in-python/>

Dunders (Double Underscores) in Python

Dunders, or double underscores, in Python are used to define special methods that allow you to customize the behavior of your classes. These methods are called dunder methods because they start and end with double underscores, such as `__init__()` and `__str__()`. Here are some key points to keep in mind when working with duunders in Python:

1. Dunder methods are used to customize the behavior of your classes, such as defining how instances are created, compared, and printed.
2. Dunder methods are called automatically by Python in response to certain events, such as creating a new instance or comparing two objects.
3. Dunder methods are defined using the `def` keyword and start and end with double underscores, such as `__init__()` and `__str__()`.
4. Dunder methods are an important part of Python's object-oriented programming model and allow you to create classes that behave in a more natural and intuitive way.

It's important to note that while duunders can make your code more powerful and flexible, they can also make it harder to understand for beginners. Use them judiciously and only when they make sense for your particular use case.

RESOURCES:

- <https://www.geeksforgeeks.org/dunder-magic-methods-python/>
- <https://www.tutorialsteacher.com/python/magic-methods-in-python>

SOAP vs REST

SOAP (Simple Object Access Protocol) and REST (Representational State Transfer) are two different approaches to building web services. Here are some key points to keep in mind when working with SOAP and REST:

1. SOAP is a protocol for exchanging structured data between applications over the internet. It uses XML (Extensible Markup Language) to define the structure of the data being exchanged and provides a standardized way of communicating between applications.
2. REST is an architectural style for building web services that uses a set of constraints to define how data is exchanged between applications. It uses HTTP (Hypertext Transfer Protocol) to exchange data and provides a more flexible and lightweight approach to building web services.

It's important to note that while SOAP and REST are different approaches to building web services, they both have their own strengths and weaknesses depending on the needs of the organization. SOAP is often used when you need a more standardized approach to exchanging data between applications, while REST is often used when you need a more flexible and lightweight approach.

RESOURCES:

- <https://www.redhat.com/en/topics/api/what-is-a-rest-api>
- <https://smartbear.com/blog/soap-vs-rest-whats-the-difference/>

MVC in Python:

MVC (Model-View-Controller) is a design pattern that is widely used in software development. While it is not a specific technology or framework, it can be implemented in many programming languages, including Python.

1. **Model:** The model represents the data and business logic of the application. In Python, the model is typically implemented using a database or an ORM (Object-Relational Mapping) library like SQLAlchemy.
2. **View:** The view represents the user interface of the application. In Python, the view is typically implemented using a web framework like Django or Flask, which provides tools for rendering HTML templates and handling user input.
3. **Controller:** The controller acts as an intermediary between the model and the view. It receives input from the user via the view, interacts with the model to retrieve or modify data, and then updates the view to display the results. In Python, the controller is

typically implemented using a web framework like Django or Flask, which provides tools for routing requests and handling user input.

Using the MVC pattern in Python can help you separate concerns and make your code more modular and maintainable. By keeping your data, business logic, and user interface separate, you can more easily modify and extend your application over time.

RESOURCES:

- <https://www.geeksforgeeks.org/mvc-design-pattern/>
- <https://realpython.com/the-model-view-controller-mvc-paradigm-summarized-with-legos/>

Data Engineering

Data engineering is the process of transforming raw data into a format that can be easily analyzed and used to make business decisions. It involves designing, building, and maintaining the infrastructure and systems that are used to collect, store, process, and analyze large amounts of data.

Data engineering is a critical component of data-driven organizations, as it enables businesses to make informed decisions based on accurate and reliable data. Data engineers work closely with data scientists, analysts, and other stakeholders to ensure that data is properly collected, stored, and processed. They are responsible for designing and building data pipelines, creating data models, and ensuring data quality.

Data engineering requires a strong understanding of database management systems, data modeling, ETL (extract, transform, load) processes, and programming languages such as Python and SQL. It also requires knowledge of big data technologies such as Hadoop, Spark, and NoSQL databases.

Overall, data engineering plays a crucial role in enabling businesses to leverage the power of data to gain insights and make informed decisions.

RESOURCES:

- <https://www.simplilearn.com/tutorials/big-data-tutorial/what-is-data-engineering>

Data Lake, Data Warehouse, and Data Lakehouse

Data lake, data warehouse, and data lakehouse are terms used in the field of data management to describe different approaches to storing and processing large amounts of data. Here are some key points to keep in mind when working with these terms:

1. A data lake is a storage repository that holds large amounts of raw data in its native format. Data lakes are designed to store vast amounts of unstructured and semi-structured data that may not fit neatly into a traditional relational database.
2. A data warehouse is a large, centralized repository of integrated data from one or more sources. Data warehouses are designed to support business intelligence (BI) activities, such as reporting, analysis, and data mining.
3. A data lakehouse is a hybrid approach that combines the features of both data lakes and data warehouses. It allows organizations to store and analyze both raw and structured data in a single platform.

It's important to note that while these terms are often used interchangeably, they represent different approaches to managing and processing data. Each approach has its own strengths and weaknesses, depending on the needs of the organization.

RESOURCES:

- <https://aws.amazon.com/big-data/datalakes-and-analytics/what-is-a-data-lake/>
- <https://www.oracle.com/pk/database/what-is-a-data-warehouse/>
- <https://www.databricks.com/glossary/data-lakehouse>

ETL and ELT

ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) are two different approaches to moving and processing data from source systems to target systems. Here are some key points to keep in mind when working with ETL and ELT:

1. ETL is a traditional approach to data integration that involves extracting data from source systems, transforming it into a format that can be used by the target system, and loading it into the target system. ETL is typically used when the target system has strict requirements for data quality and consistency.
2. ELT is a newer approach to data integration that involves extracting data from source systems and loading it into the target system without transforming it. The transformation is then performed in the target system using tools like SQL or other programming languages. ELT is typically used when the target system has more flexible data requirements and can handle unstructured or semi-structured data.

It's important to note that while ETL and ELT are different approaches to data integration, they both have their own strengths and weaknesses depending on the needs of the organization.

ETL is often used when data quality is a top priority, while ELT is often used when flexibility and agility are more important.

RESOURCES:

- <https://aws.amazon.com/compare/the-difference-between-etl-and-elt/>
- <https://www.glik.com/us/etl/etl-vs-elt>
- <https://www.ibm.com/blog/elt-vs-etl-whats-the-difference/>

Orchestration

Orchestration is the process of automating and coordinating complex workflows and systems. It involves defining the sequence of steps required to complete a task or process, and automating the execution of those steps. Here are some key points to keep in mind when working with orchestration:

1. Orchestration is often used in the context of large-scale distributed systems, such as cloud computing environments, where there are multiple components that need to work together.
2. Orchestration involves defining the sequence of steps required to complete a task or process, and automating the execution of those steps.
3. Orchestration tools and platforms typically provide a visual interface for defining workflows and automating tasks.
4. Orchestration can help improve efficiency, reduce errors, and increase reliability by automating complex processes and reducing the need for manual intervention.

It's important to note that while orchestration can be a powerful tool for automating complex workflows and systems, it can also introduce new complexities and challenges. It's important to carefully design and test orchestration workflows to ensure that they are reliable and efficient.

RESOURCES:

- <https://www.databricks.com/glossary/orchestration>
- <https://www.bmc.com/blogs/it-orchestration-vs-automation-whats-the-difference/>

What Is Machine Learning?

Machine Learning is a branch of Artificial Intelligence that allows machines to learn and improve from experience automatically. It is defined as the field of study that gives computers the capability to learn without being explicitly programmed. It is quite different than traditional programming.

Resources:

- <https://www.simplilearn.com/tutorials/machine-learning-tutorial>

How Machine Learning Works?

Machine Learning is a core form of Artificial Intelligence that enable machine to learn from past data and make predictions

It involves data exploration and pattern matching with minimal human intervention. There are mainly four technologies that machine learning used to work:

1. Supervised Learning:

Supervised Learning is a machine learning method that needs supervision similar to the student-teacher relationship. In supervised Learning, a machine is trained with well-labeled data, which means some data is already tagged with correct outputs. So, whenever new data is introduced into the system, supervised learning algorithms analyze this sample data and predict correct outputs with the help of that labeled data.

It is classified into two different categories of algorithms. These are as follows:

- **Classification:** It deals when output is in the form of a category such as Yellow, blue, right or wrong, etc.
- **Regression:** It deals when output variables are real values like age, height, etc.

This technology allows us to collect or produce data output from experience. It works the same way as humans learn using some labeled data points of the training set. It helps in optimizing the performance of models using experience and solving various complex computation problems.

RESOURCES:

- <https://www.datacamp.com/blog/supervised-machine-learning>

2. Unsupervised Learning:

Unlike supervised learning, unsupervised Learning does not require classified or well-labeled data to train a machine. It aims to make groups of unsorted information based on some patterns and differences even without any labeled training data. In unsupervised Learning, no supervision is provided, so no sample data is given to the machines. Hence, machines are restricted to finding hidden structures in unlabeled data on their own.

It is classified into two different categories of algorithms. These are as follows:

- **Clustering:** It deals when there is a requirement of inherent grouping in training data, e.g., grouping students by their area of interest.
- **Association:** It deals with the rules that help to identify a large portion of data, such as students who are interested in ML and also interested in AI.

RESOURCES:

- <https://towardsdatascience.com/unsupervised-learning-algorithms-cheat-sheet-d391a39de44a>
- <https://www.guru99.com/unsupervised-machine-learning.html>
- <https://www.seldon.io/supervised-vs-unsupervised-learning-explained#:~:text=The%20main%20difference%20between%20supervised,processes%20unlabelled%20or%20raw%20data.>

3. Semi-supervised learning:

Semi-supervised Learning is defined as the combination of both supervised and unsupervised learning methods. It is used to overcome the drawbacks of both supervised and unsupervised learning methods.

In the semi-supervised learning method, a machine is trained with labeled as well as unlabeled data. Although, it involves a few labeled examples and a large number of unlabeled examples.

Speech analysis, web content classification, protein sequence classification, and text documents classifiers are some most popular real-world applications of semi-supervised Learning.

Importance of Machine Learning

Although machine learning is continuously evolving with so many new technologies, it is still used in various industries.

Machine learning is important because it gives enterprises a view of trends in customer behavior and operational business patterns, as well as supports the development of new products. Many of today's leading companies, such as **Facebook**, **Google**, and **Uber**, make machine learning a central part of their operations. Machine learning has become a significant competitive differentiator for many companies.

Machine learning has several practical applications that drive the kind of real business results - such as time and money savings - that have the potential to dramatically impact the future of your organization. In particular, we see tremendous impact occurring within the customer care industry, whereby machine learning is allowing people to get things done more quickly and efficiently. Through Virtual Assistant solutions, machine learning automates tasks that would otherwise need to be performed by a live agent - such as changing a password or checking an account balance. This frees up valuable agent time that can be used to focus on the kind of customer care that humans perform best: high touch, complicated decision-making that is not as easily handled by a machine. At Interactions, we further improve the process by eliminating the decision of whether a request should be sent to a human or a machine: unique Adaptive Understanding technology, the machine learns to be aware of its limitations, and bailout to humans when it has low confidence in providing the correct solution.

Use cases of Machine Learning Technology

Machine Learning is broadly used in every industry and has a wide range of applications, especially that involves collecting, analyzing, and responding to large sets of data. The importance of Machine Learning can be understood by these important applications.

Some important applications in which machine learning is widely used are given below:

1. **Healthcare:** Machine Learning is widely used in the healthcare industry. It helps healthcare researchers to analyze data points and suggest outcomes. Natural language processing helped to give accurate insights for better results of patients. Further, machine learning has improved the treatment methods by analyzing external data on patients' conditions in terms of X-ray, Ultrasound, CT-scan, etc. NLP, medical imaging, and genetic information are key areas of machine learning that improve the diagnosis, detection, and prediction system in the healthcare sector.
2. **Automation:** This is one of the significant applications of machine learning that helps to make the system automated. It helps machines to perform repetitive tasks without human intervention. As a machine learning engineer and data scientist, you have the responsibilities to solve any given task multiple times with no errors. However, this is not practically possible for humans. Hence machine learning has developed various models to automate the process, having the capability of performing iterative tasks in lesser time.
3. **Image Recognition:** It is one of the most common applications of machine learning which is used to detect the image over the internet. Further, various social media sites such as Facebook uses image recognition for tagging the images to your Facebook friends with its feature named auto friend tagging suggestion.

Further, now a day's, almost all mobile devices come with exciting face detection features. Using this feature, you can secure your mobile data with face unlocking, so if anyone tries to access your mobile device, they cannot open without face recognition.
4. **Speech Recognition:** Speech recognition is one of the biggest achievements of machine learning applications. It enables users to search content without writing text or, in other words, 'search by voice'. It can search content/products on YouTube, Google, Amazon, etc. platforms by your voice. This technology is

referred to as speech recognition.

It is a process of converting voice instructions into text; hence it is also known as 'Speech to text' or 'Computer speech recognition. Some important examples of speech recognition are **Google assistant, Siri, Cortana, Alexa**, etc.

5. **Product Recommendation:** It is one of the biggest achievements made by machine learning which helps various e-commerce and entertainment companies like Flipkart, Amazon, Netflix, etc., to digitally advertise their products over the internet. When anyone searches for any product, they start getting an advertisement for the same product while internet surfing on the same browser. This is possible by machine learning algorithms that work on users' interests or past experience and accordingly recommend them for products. For e.g., when we search for a laptop on the Amazon platform, then it also gets started with so many other laptops having the same categories and criteria. Similarly, when we use Netflix, we find some recommendations for entertainment series, movies, etc. Hence, this is also possible by machine learning algorithms.
6. **Language Translation:** The use of Machine learning can be seen in language translation. It uses the sequence-to-sequence learning algorithms for translating one language into other. Further, it also uses images recognition techniques to identify the text from one language to other. Similarly, Google's GNMT (Google Neural Machine Translation) provides this feature, which is a Neural Machine Learning that translates the text into our familiar language, and it is called automatic translation.

MATHEMATICS FOR ML

Linear Algebra

Linear algebra is a fundamental mathematical tool for machine learning. It involves the study of vectors, matrices, and their operations. In ML, it helps with tasks like dimensionality reduction, regression, and classification. By utilizing concepts such as matrix multiplication and eigenvalue

decomposition, practitioners can analyze and manipulate data effectively. Understanding linear algebra is crucial for developing and implementing ML algorithms.

Resources:

- https://www.youtube.com/watch?v=e9h-ZZ_ahRg&list=PLKnIA16_RmvYu0fS_RuIB2eTbJcTFdrAA&pp=iAQB

Probability And Statistic

Probability and statistics are essential components of machine learning (ML). Probability theory helps quantify uncertainty, while statistics enables data analysis and inference. They aid in tasks like data preprocessing, model evaluation, and understanding results. Understanding probability and statistics is crucial for making informed decisions and building accurate ML models.

Resources:

- https://www.youtube.com/watch?v=tPhzDKjQBpo&list=PLKnIA16_RmVbVrE0eZO2bCaFln6jaNq-1&pp=iAQB
- <https://www.youtube.com/watch?v=DUT4WEUngt0>
- <https://www.youtube.com/watch?v=ndHDsvqmbul>

SUPERVISED LEARNING ALGORITHMS

Regression:

Regression in machine learning refers to a supervised learning task where the goal is to predict a continuous output variable based on input features. It involves finding the mathematical relationship between the input variables and the output variable in order to make accurate predictions. In regression, the model learns from labeled training data to estimate the relationship between the input variables and the target variable, allowing it to make predictions for new, unseen data. The output variable in regression can take on any numerical value within a specific range, making it different from classification, which involves predicting discrete categories.

Resources:

- <https://www.javatpoint.com/regression-analysis-in-machine-learning>

- <https://towardsdatascience.com/a-beginners-guide-to-regression-analysis-in-machine-learning-8a828b491bbf>

Linear Regression:

Linear regression is a simple and widely used algorithm for predicting continuous numeric values. It assumes a linear relationship between the input features and the target variable. In simple linear regression, there is only one input feature, while multiple linear regression involves multiple input features. The goal is to find the best-fit line that minimizes the difference between the predicted and actual values. Feature scaling techniques like standardization or normalization may be used to ensure that all input features contribute equally.

Resources:

- <https://machinelearningmastery.com/linear-regression-for-machine-learning/>
- <https://machinelearningmastery.com/overfitting-and-underfitting-with-machine-learning-algorithms/>
- https://drive.google.com/file/d/17O3BYKMy6EX9RAi2ZgOznUju02hUmq_Y/view?usp=drive_link
- https://drive.google.com/file/d/19SMfbtP3hpedMHwrvS2qM3UVZGM-51pU/view?usp=drive_link
- https://www.youtube.com/watch?v=UZPfbG0jNec&list=PLKnIA16_Rmva-wY_HBh1gTH32ocu2SoTr

Notebook link:

- [REGRESSION MASTER NOTEBOOK | Kaggle](#)

CLASSIFICATION

Classification is a supervised machine learning method where the model tries to predict the correct label of a given input data. In classification, the model is fully trained using the training data, and then it is evaluated on test data before being used to perform prediction on new unseen data.

Resources:

- <https://machinelearningmastery.com/types-of-classification-in-machine-learning/>
- <https://www.simplilearn.com/regression-vs-classification-in-machine-learning-article>

- <https://www.springboard.com/blog/data-science/regression-vs-classification/#:~:text=The%20most%20significant%20difference%20between,types%20of%20machine%20learning%20algorithms.>

Logistic Regression:

Logistic regression is a classification algorithm used when the target variable is binary or categorical. It models the relationship between the input features and the probability of belonging to a certain class. Logistic regression uses a logistic function (sigmoid function) to map the output to a probability value between 0 and 1. The decision boundary is typically set at a threshold value (e.g., 0.5) to classify instances into different classes. Logistic regression can be extended to handle multi-class classification using techniques like one-vs-rest or softmax regression. Evaluation metrics like accuracy, precision, recall, and F1-score are used to assess the performance of logistic regression models.

Resources:

- <https://towardsdatascience.com/logistic-regression-detailed-overview-46c4da4303bc>
- https://www.youtube.com/watch?v=XXNzVftWGY&list=PLKnIA16_RmVb-ZTsM1QS-tlwmlkeGSnru
- https://drive.google.com/file/d/1w0HrY0yVp_m3PuNWQvTTICW4n3KPqWGx/view?usp=drive_link

Naive Bayes:

Naive Bayes is a classification algorithm in machine learning that operates on the principles of Bayes' theorem with an assumption of feature independence. Despite its simplistic assumption, Naive Bayes has gained popularity due to its efficiency and effectiveness in various real-world applications. It calculates the probability of a data instance belonging to a specific class based on the probabilities of its individual features. By estimating prior probabilities and conditional probabilities during training, Naive Bayes can predict the class label of new instances by calculating posterior probabilities. This algorithm is widely used in text classification, spam filtering, sentiment analysis, and other tasks involving high-dimensional data.

Resources:

- <https://www.javatpoint.com/machine-learning-naive-bayes-classifier>
- <https://www.turing.com/kb/an-introduction-to-naive-bayes-algorithm-for-beginners>
- https://scikit-learn.org/stable/modules/naive_bayes.html
- https://drive.google.com/file/d/19SMfbtP3hpedMHwrvS2qM3UVZGM-51pU/view?usp=drive_link

- https://www.youtube.com/watch?v=Ty7knppVo9E&list=PLKnIA16_RmvZ67wQaHoBuzXaDAfPz-a6l

Notebook link:

- <https://www.kaggle.com/code/prashant111/naive-bayes-classifier-in-python>

Decision Trees:

Decision trees are a popular and intuitive machine learning algorithm used for **both classification and regression tasks**. They create a hierarchical structure of decision nodes that represent features and splitting criteria to make predictions. Each decision node in the tree partitions the data based on specific feature conditions until a leaf node is reached, which provides the final prediction. Decision trees are advantageous as they are easy to understand, interpret, and visualize. They can handle both categorical and numerical data, as well as missing values. Decision trees also support feature importance analysis, allowing insights into the most influential variables. However, decision trees are prone to overfitting, which can be mitigated using techniques like pruning and ensemble methods. Decision trees are widely applied in various domains, including finance, healthcare, and customer analytics, due to their simplicity, interpretability, and ability to handle both structured and unstructured data.

Resources:

- <https://towardsdatascience.com/decision-tree-in-machine-learning-e380942a4c96>
- <https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm>
- <https://scikit-learn.org/stable/modules/tree.html>
- https://www.youtube.com/watch?v=qwgmSSTdiXs&list=PLKnIA16_RmvYGY_n9PP8zN-OLG9MoZRjU
[Decision Trees - Introduction and Geometric Intuition](#)

Notebook link:

- <https://www.kaggle.com/code/satishgunjal/tutorial-decision-tree>

Random Forest:

Random Forest is a popular and intuitive machine learning algorithm used for **both classification and regression tasks**; it is a powerful ensemble learning algorithm that combines multiple decision trees to make accurate predictions. It reduces overfitting by training each tree on random subsets of the data and features. Random Forest is robust, handles high-dimensional data well, and provides insights into important features. It finds applications in finance, healthcare, and image recognition, making it a popular algorithm in machine learning.

Resources:

- <https://www.turing.com/kb/random-forest-algorithm>

- <https://towardsdatascience.com/understanding-random-forest-58381e0602d2>
- <https://www.javatpoint.com/machine-learning-random-forest-algorithm>
- <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- https://www.youtube.com/watch?v=bHK1fE_BUms&list=PLKnIA16_RmvZyqP3WGUo7iVzillea_1bp

Notebook link:

- <https://www.kaggle.com/code/prashant111/random-forest-classifier-tutorial>

SVM:

Support Vector Machines (SVM) is a powerful machine learning algorithm used for **both classification and regression tasks**. SVMs excel in solving complex problems by creating optimal decision boundaries between different classes. By finding the best hyperplane that maximally separates data points, SVMs can effectively handle both linearly separable and non-linearly separable datasets. SVMs have gained popularity due to their ability to handle high-dimensional data and their robustness against overfitting. They have been successfully applied in various domains, including image recognition, text categorization, and bioinformatics. With their versatility and strong theoretical foundations, SVMs are widely recognized as a valuable tool in the field of machine learning.

Resources:

- <https://www.javatpoint.com/machine-learning-support-vector-machine-algorithm>
- <https://towardsdatascience.com/support-vector-machine-introduction-to-machine-learning-algorithms-934a444fca47>
- https://drive.google.com/file/d/1Vq8OD5a6cWeoQb-46g6a_2wSMm2lr83V/view?usp=drive_link
- https://www.youtube.com/watch?v=uqTxMLjLS8M&list=PLKnIA16_RmvbOIFee-ra7U6jR2olbCZBL

Notebook link:

- <https://www.kaggle.com/code/prashant111/svm-classifier-tutorial>

KNN:

K-Nearest Neighbors (KNN) is a versatile and intuitive machine learning algorithm used for classification and regression. It predicts the label or value of a data point based on the majority or average of its nearest neighbors in the feature space. KNN is non-parametric and can handle both linear and nonlinear relationships in the data. Although it can be computationally intensive,

KNN is popular for its simplicity and effectiveness, especially in complex decision boundary scenarios or when data distributions are unclear.

Resources:

- <https://towardsdatascience.com/machine-learning-basics-with-the-k-nearest-neighbors-algorithm-6a6e71d01761>
- <https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning>
- <https://stats.stackexchange.com/questions/364351/regression-knn-model-vs-classification-knn-model>
- https://drive.google.com/file/d/1ljinOURySxxoIktttdtg3rzOydlbNSder/view?usp=drive_link
- https://drive.google.com/file/d/13eSND_GH39d3Ek4o_S0_djv1Ftmp-4hW/view?usp=drive_link
- https://www.youtube.com/watch?v=BYaoDZM1IcU&list=PLKnIA16_RmvZiE-IEdN5RDi18-u-T43zd

Notebook link:

- <https://www.kaggle.com/code/nebilekodaz/knn-notebook>

Practical Assignment:

Build a decision tree classifier using scikit-learn and evaluate its performance on a dataset of customer churn prediction. Use Python and scikit-learn to preprocess the data, train the decision tree model, and evaluate its accuracy, precision, recall, and F1-score.

UNSUPERVISED LEARNING ALGORITHM

K-Mean Clustering:

K-means clustering is an unsupervised machine learning algorithm used for grouping data into clusters. It assigns data points to clusters based on their proximity to centroid points. The algorithm iteratively updates the centroids until convergence, creating distinct clusters. K-means is widely used for exploratory data analysis and segmentation tasks, although it has assumptions and limitations. It is popular due to its simplicity and scalability for clustering large datasets.

Resources:

- <https://www.javatpoint.com/k-means-clustering-algorithm-in-machine-learning>
- <https://towardsdatascience.com/understanding-k-means-clustering-in-machine-learning-6a6e67336aa1>

- <http://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>
- https://www.youtube.com/watch?v=5shTLzwAdEc&list=PLKnIA16_RmVbA_hYXIRgdCg9bn8ZQK2z9

PCA:

Principal Component Analysis (PCA) is a widely used dimensionality reduction technique in machine learning and data analysis. It aims to transform high-dimensional data into a lower-dimensional representation while preserving the most important information. By identifying the principal components that capture the maximum variance in the data, PCA allows for simplification, visualization, and improved computational efficiency. It is particularly useful when dealing with datasets containing a large number of features or when visualizing data in lower-dimensional spaces. PCA has applications in various fields, including image processing, signal processing, and pattern recognition. Its ability to uncover underlying patterns and reduce redundancy makes PCA a valuable tool for data exploration and analysis.

Resources:

- [https://www.keboola.com/blog/pca-machine-learning#:~:text=Principal%20Component%20Analysis%20\(PCA\)%20is,%2Dnoising%2C%20and%20plenty%20more.](https://www.keboola.com/blog/pca-machine-learning#:~:text=Principal%20Component%20Analysis%20(PCA)%20is,%2Dnoising%2C%20and%20plenty%20more.)
- [A Guide to Principal Component Analysis \(PCA\) for Machine Learning.](#)
- [Principal Component Analysis in Machine Learning | Simplilearn](#)
- [Principal Component Analysis in Machine Learning | PCA in ML](#)
- https://drive.google.com/file/d/1liLkl-ez_UfKzz8gmYtjzmF-0wjN7wVP/view?usp=drive_link
- <https://www.geeksforgeeks.org/principal-component-analysis-with-python/>
- https://drive.google.com/file/d/13eSND_GH39d3Ek4o_S0_djv1Ftmp-4hW/view?usp=drive_link
- https://www.youtube.com/watch?v=Y8Yu4sNSWkY&list=PLKnIA16_RmVb-1-kGcacbQE13xrVWQjEo

Notebook link:

- <https://www.kaggle.com/code/ryanholbrook/principal-component-analysis>

Practical Assignment:

Apply K-means clustering to a customer segmentation dataset and visualize the clusters using matplotlib or seaborn. Identify meaningful clusters and analyze the characteristics of each cluster to gain insights into customer behavior.

Deep Learning

Artificial Neural Networks (ANN):

Artificial neural networks are models inspired by the structure and function of biological neurons. This subtopic introduces the basics of neural networks, including neurons, activation functions (e.g., sigmoid, tanh, and ReLU), and the backpropagation algorithm for training neural networks. You'll learn about feedforward networks, backward propagation of errors, and techniques to avoid issues like vanishing and exploding gradients.

- [What are Neural Networks? | IBM](#)
- [What is a Neural Network? - Artificial Neural Network Explained - AWS](#)
- [What Is a Neural Network?](#)
- [Neural networks - YouTube](#)
- [Deep Learning Fundamentals - Intro to Neural Networks - YouTube](#)

Neurons:

A neural network works similarly to the human brain's neural network. A “neuron” in a neural network is a mathematical function that collects and classifies information according to a specific architecture.

- [Neurons in Neural Networks | Baeldung on Computer Science](#)

MLP

Multi layer perceptron (MLP) is a supplement of feed forward neural network. It consists of three types of layers—the input layer, output layer and hidden layer, as shown in Fig. 3. The input layer receives the input signal to be processed.

Resources:

- https://drive.google.com/file/d/1_aEslSR7h-DclXoGgcQx-rlpwWjATgwq/view?usp=drive_link
- https://drive.google.com/file/d/1rBLr-Eo6KhpD0KsmSnGChPCpoN5wsSYV/view?usp=drive_link

Feed Forward neural networks

Feed forward neural networks are artificial neural networks in which nodes do not form loops. This type of neural network is also known as a multi-layer neural network as all information is only passed forward. During data flow, input nodes receive data, which travel through hidden layers, and exit output nodes.

Resources:

- [Basic Introduction to Feed-Forward Network in Deep Learning](#)
- https://drive.google.com/file/d/184NNHHpjq--IYEsN6gf70VTXr0gkjsz/view?usp=drive_link

Backpropagation

Backpropagation is the essence of neural network training. It is the method of fine-tuning the weights of a neural network based on the error rate obtained in the previous epoch (i.e., iteration). Proper tuning of the weights allows you to reduce error rates and make the model reliable by increasing its generalization.

Resources:

- [Back Propagation in Neural Network: Machine Learning Algorithm](#)
- https://drive.google.com/file/d/184NNHHpjq--IYEsN6gf70VTXr0gkjsz/view?usp=drive_link

Hyper Parameters

Hyperparameters are parameters whose values are set before starting the model training process. Deep learning models, including convolutional neural network (CNN) and recurrent neural network (RNN) models can have anywhere from a few hyperparameters to a few hundred hyperparameters.

Resources:

- [What are Hyperparameters ? and How to tune the Hyperparameters in a Deep Neural Network? | by Pranoy Radhakrishnan | Towards Data Science](#)
- [Tuning the Hyperparameters and Layers of Neural Network Deep Learning](#)
- https://drive.google.com/file/d/1rBLr-Eo6KhpD0KsmSnGChPCpoN5wsSYV/view?usp=drive_link

Gradient Descent

Gradient descent is an optimization algorithm which is commonly-used to train machine learning models and neural networks.

Resources:

- [What is Gradient Descent? | IBM.](#)

Overfitting

Overfitting occurs when the model has a high variance, i.e., the model performs well on the training data but does not perform accurately in the evaluation set. The model memorizes the data patterns in the training dataset but fails to generalize to unseen examples

Resources:

- https://drive.google.com/file/d/1rBLr-Eo6KhpD0KsmSnGChPCpoN5wsSYV/view?usp=drive_link

Regularization

Process of introducing additional information in order to solve an ill-posed problem or to prevent overfitting

Resources:

- https://drive.google.com/file/d/1rBLr-Eo6KhpD0KsmSnGChPCpoN5wsSYV/view?usp=drive_link

Optimization

Gradient Descent Optimization Algos

1. Nesterov accelerated gradient
2. Adagrad
3. Adadelta
4. RMSprop
5. Adam
6. AdaMax
7. NAdam
8. AMSGrad

Resources:

- [An overview of gradient descent optimization algorithms](#)
- https://drive.google.com/file/d/19OP7zrBJJSKI2xrPyoho2JVI_b07Dw-v/view?usp=drive_link

Weight Initialization

Weight initialization is a procedure to set the weights of a neural network

Resources:

- [How to Initialize Weights in Neural Networks?](#)
- [Weight Initialization Techniques in Neural Networks | by Saurabh Yadav | Towards Data Science](#)
- https://drive.google.com/file/d/1_3V1xQGNzoO4dhCeDvvKHkYGN0FMOvGf/view?usp=drive_link

Learning Rate

The learning rate is a hyperparameter that controls how much to change the model in response to the estimated error each time the model weights are updated.

Resources:

- [What is Learning Rate in Machine Learning | Deepchecks](#)
- https://drive.google.com/file/d/1_3V1xQGNzoO4dhCeDvvKHkYGN0FMOvGf/view?usp=drive_link

Batch Normalization

Batch Norm is a normalization technique done between the layers of a Neural Network instead of in the raw data. It is done along mini-batches instead of the full data set. It serves to speed up training and use higher learning rates, making learning easier. the standard deviation of the neurons' output.

Resources:

- [Introduction to Batch Normalization](#)
- https://drive.google.com/file/d/1_3V1xQGNzoO4dhCeDvvKHkYGN0FMOvGf/view?usp=drive_link

Loss Function:

A loss function is a function that compares the target and predicted output values; measures how well the neural network models the training data. When training, we aim to minimize this loss between the predicted and target outputs

Resources:

- [Understanding Loss Function in Deep Learning - Analytics Vidhya](#)
- [Loss Functions and Their Use In Neural Networks | by Vishal Yathish | Towards Data Science.](#)
- [Session On Different Types Of Loss Function In Deep Learning](#)

Neural networks Practice Project

[Titanic: Neural Network for Beginners | Kaggle](#)

Convolutional Neural Networks (CNN)

Convolutional Neural Networks are specialized neural networks for processing grid-like input data, such as images. CNNs use convolutional layers, pooling layers, and activation functions to extract spatial hierarchies of features from the input data. Convolutional filters slide across the input data, performing element-wise multiplications and aggregating the results. Pooling layers reduce the spatial dimensions of the features, preserving the most important information. Activation functions like ReLU introduce non-linearities, enabling CNNs to learn complex patterns and capture spatial relationships. CNNs are widely used in image classification, object detection, and image segmentation tasks.

Resources:

- [Convolutional Neural Networks, Explained | by Mayank Mishra | Towards Data Science](#)
- [Introduction to Convolutional Neural Networks \(CNN\)](#)
- [Convolutional Neural Network Tutorial \[Update\]](#)
- https://drive.google.com/file/d/19M70o9PpnVUrw71bMadh6rq4j7TKMRMM/view?usp=drive_link
- https://drive.google.com/file/d/184NNHHpig--IYEsN6gf70VTXr0gkjsz/view?usp=drive_link
- [Deep Learning Fundamentals - Intro to Neural Networks - YouTube](#)

AutoEncoders

An autoencoder is a type of artificial neural network used to learn efficient codings of unlabeled data (unsupervised learning). An autoencoder learns two functions: an encoding function that transforms the input data, and a decoding function that recreates the input data from the encoded representation.

Resources:

- [Introduction to autoencoders.](#)
- [Autoencoders in Deep Learning: Tutorial & Use Cases \[2023\]](#)
- [Intro to Autoencoders | TensorFlow Core](#)
- https://drive.google.com/file/d/1liLkl-ez_UfKzz8gmYtjzmF-0wjN7wVP/view?usp=drive_link

Generative Adversarial Networks

Generative Adversarial Networks (GANs) are a powerful class of neural networks that are used for [unsupervised learning](#). It was developed and introduced by Ian J. Goodfellow in 2014. GANs are basically made up of a system of two competing neural network models which compete with each other and are able to analyze, capture and copy the variations within a dataset.

Resources:

- [An End-to-End Introduction to Generative Adversarial Networks\(GANs\)](#)
- [Generative Adversarial Network \(GAN\) - GeeksforGeeks](#)
- [Overview of GAN Structure | Machine Learning | Google for Developers](#)
- [What are Generative Adversarial Networks \(GANs\) | Simplilearn](#)
- [Intro to Generative Adversarial Networks \(GANs\) - YouTube](#)

CNN Compression

Nowadays, Convolutional Neural Networks are used extensively for Image Classification, Facial and Object Recognition, Medical Imaging Analysis, etc. Also, there are applications like Self Driving Cars, where predictions are desired in real-time with low latency. The current neural network architectures have millions of parameters, and it becomes a problem to deploy such heavy models on IoT devices and other edge devices with memory restrictions. Compression Techniques would help us solve these problems by reducing the size of CNN models obtained by minimizing the number of parameters that help to reduce the complexity of these models. With fewer parameters, the inference time for predictions would be lesser than the original

Convolutional Neural Networks model. The training time for CNN models will be lesser than the original models.

Resources:

- [Compression Techniques for Convolutional Neural Networks](#)

Recurrent Neural Networks (RNN)

Recurrent Neural Networks are designed to process sequential data, where the current input depends on the previous inputs. RNNs have feedback connections that allow them to capture temporal dependencies in the data. Each RNN unit maintains a hidden state that captures information from previous inputs. The hidden state is updated at each time step, incorporating new information and passing it to the next time step. RNNs suffer from the vanishing gradient problem, which limits their ability to capture long-term dependencies. Variations like LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Units) address this problem by introducing additional gating mechanisms. RNNs are widely used in tasks like natural language processing, speech recognition, and time series analysis.

Resources:

- [A Brief Overview of Recurrent Neural Networks \(RNN\) - Analytics Vidhya](#)
- [A Guide to Recurrent Neural Networks: Understanding RNN and LSTM Networks](#)
- [An Introduction to Recurrent Neural Networks and the Math That Powers Them - MachineLearningMastery.com](#)

Fine Tuning & Transfer Learning:

Fine-tuning is a [training technique](#) that consists of the reuse of predefined and pre-trained CNN (convolutional neural network) architectures.

Resources:

- [Fine-tuning a Neural Network explained - deeplizard](#)
- [CNN Transfer Learning & Fine Tuning | by Victor Roman | Towards Data Science](#)
- [TRANSFER LEARNING AND FINE TUNING OF NEURAL NETWORKS](#)
- [What Is Fine-Tuning and How Does It Work in Neural Networks?](#)

Practical Assignment:

Build a CNN model using TensorFlow or PyTorch for image classification on the CIFAR-10 dataset. Preprocess the dataset, design the CNN architecture, train the model using backpropagation, and evaluate its performance using accuracy metrics.

Model Evaluation and Deployment

Model Evaluation Techniques:

Cross-validation is a technique for assessing the performance and generalization ability of machine learning models. It involves splitting the data into multiple folds, training the model on a subset of folds, and evaluating it on the remaining fold. This process is repeated multiple times, and the performance metrics are averaged. Cross-validation helps to estimate the model's performance on unseen data and provides insights into its robustness. Hyperparameter tuning involves optimizing the model's hyperparameters to achieve the best performance. Techniques like grid search or random search can be used to explore different hyperparameter combinations and find the optimal values. Evaluation metrics like accuracy, precision, recall, F1-score, and receiver operating characteristic (ROC) curve provide a quantitative assessment of the model's performance.

Model Deployment:

After training and evaluating a machine learning model, it needs to be deployed in real-world applications. Model deployment involves exporting and serializing the trained model to a file format that can be easily loaded and used for predictions. Web frameworks like Flask or Django can be used to create APIs (Application Programming Interfaces) for serving the model as a web service. These frameworks handle HTTP requests and responses, allowing users to interact with the model through RESTful APIs. Containerization using Docker is a technique to package the model along with its dependencies and runtime environment. Docker containers provide a consistent and portable environment for running the model, making it easier to deploy and scale in different deployment environments.

Practical Assignment:

Evaluate and fine-tune a machine learning model using cross-validation and hyperparameter tuning. Deploy the optimized model as a web service using Flask or Django. Create a RESTful API that accepts input data, performs predictions using the deployed model, and returns the results to the client.

Advanced Topics

Natural Language Processing (NLP)

Natural Language Processing focuses on enabling machines to understand and process human language. It involves tasks like tokenization (splitting text into words or tokens), stemming (reducing words to their base or root form), and lemmatization (reducing words to their dictionary form). Text classification involves assigning predefined categories or labels to text documents based on their content. Sentiment analysis aims to determine the sentiment or emotion expressed in a piece of text. Named Entity Recognition (NER) involves identifying and classifying named entities like persons, organizations, locations, etc., in text. Word embeddings capture semantic relationships between words by representing them as dense vectors in a high-dimensional space. Popular word embedding models include Word2Vec, GloVe, and BERT.

Resources:

- [What is NLP: An Introductory Tutorial to Natural Language Processing \[Updated\]](#)
- [What is Natural Language Processing? | IBM](#)
- [Natural Language Processing \(NLP\) with Deep NLP from Zero to Hero | A Deep Dive into NLP with Python | Sequence Models | NLP in Hindi/Urdu - YouTube](#)
- [NLP Slides](#)

Text Processing:

Tokenization: Splitting text into words, sentences, or other meaningful units.

Stop Word Removal: Removing common words (e.g., "a", "the") that don't carry much meaning.

Stemming and Lemmatization: Reducing words to their root form.

Part-of-Speech (POS) Tagging: Assigning grammatical tags to words.

- [Introduction to Natural Language Processing for Text | by Ventsislav Yordanov | Towards Data Science](#)
- [Essential Text Pre-processing Techniques for NLP! - Analytics Vidhya](#)

Language Modeling:

N-grams: Representing sequences of words.

Statistical Language Models: Estimating the likelihood of word sequences.

Neural Language Models: Using neural networks to predict the next word.

- <https://www.analyticsvidhya.com/blog/2022/01/building-language-models-in-nlp/>

Sentiment Analysis:

Classifying the sentiment (positive, negative, neutral) of text.

Building a sentiment analysis model using machine learning or deep learning techniques.

- <https://www.analyticsvidhya.com/blog/2021/06/nlp-sentiment-analysis/>

Named Entity Recognition (NER):

Identifying and classifying named entities (e.g., person, organization, location) in text.

Using techniques like rule-based methods, conditional random fields (CRF), or deep learning models.

- [A Beginner's Introduction to NER \(Named Entity Recognition\)](#)
- [Named Entity Recognition with NLTK and SpaCy | by Susan Li | Towards Data Science](#)

Topic Modeling:

Discovering the main topics or themes in a collection of documents.

Popular techniques include Latent Dirichlet Allocation (LDA) and Non-Negative Matrix Factorization (NMF).

- [Topic Modelling in Natural Language Processing](#)

Word Embeddings:

Representing words as dense vectors in a continuous vector space. Learning word embeddings using methods like Word2Vec or GloVe.

Text Classification:

Assigning predefined categories or labels to text. Building classification models using techniques like Naive Bayes, Support Vector Machines (SVM), or deep learning models.

Sequence-to-Sequence Models:

Generating text or translating between languages using models like Recurrent Neural Networks (RNNs) or Transformer models.

Question Answering:

Building systems that can answer questions based on a given context. Techniques like extractive (finding relevant passages) or generative (generating answers) methods.

Advanced NLP Techniques:

Coreference Resolution: Resolving pronouns to their referents.

Machine Translation: Translating text between different languages.

Text Summarization: Generating concise summaries of longer text.

Dialog Systems: Building conversational agents for interactive communication.

Deep Learning for NLP:

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks for sequence modeling.

Transformer models (e.g., BERT, GPT) for various NLP tasks.

Transfer learning and fine-tuning pre-trained language models.

Resources and Tools:

Python libraries: NLTK, spaCy, scikit-learn, TensorFlow, PyTorch, Hugging Face Transformers.

Datasets and corpora for NLP research and development.

Online courses, tutorials, and books for further learning.

As you progress from beginner to expert level, make sure to gain practical experience by working on NLP projects and experimenting with different techniques and models. Stay updated with the latest research and developments in the field as NLP is an active area of research and new techniques and models are constantly emerging

Computer Vision

Computer vision is an interdisciplinary field that focuses on enabling computers to extract information from visual data, such as images or videos, in a way similar to human vision. It encompasses a wide range of tasks, including image classification, object detection, segmentation, tracking, and image generation. Through the use of advanced algorithms, machine learning, and deep learning techniques, computer vision algorithms can analyze and interpret visual data to understand and extract meaningful insights. Computer vision finds applications in diverse areas, such as autonomous vehicles, medical imaging, facial recognition, surveillance, augmented reality, and robotics. By providing machines with the ability to "see" and interpret visual information, computer vision plays a vital role in revolutionizing industries and enhancing human-computer interaction.

OpenCV:

OpenCV (Open Source Computer Vision Library) is a widely used open-source library for computer vision and image processing. It offers a comprehensive set of tools and functions for handling and manipulating visual data, making it easier to develop sophisticated computer vision applications. OpenCV is written in C++ but provides interfaces for multiple programming languages, making it versatile and accessible. With its extensive range of algorithms and pre-trained models, OpenCV simplifies tasks such as image filtering, object recognition, and camera calibration. It is widely adopted in various fields, including robotics, surveillance, and medical imaging, for its reliability and performance.

Resources::

- https://docs.opencv.org/4.x/d9/df8/tutorial_root.html

Segmentation:

Segmentation in computer vision is the process of dividing an image or video into meaningful regions or objects. It aims to identify and separate different entities or regions of interest within an image or video frame. Image segmentation plays a crucial role in various computer vision tasks, including object recognition, tracking, and scene understanding. It enables machines to distinguish and extract specific regions or objects, facilitating further analysis and interpretation. Segmentation techniques can be based on different principles, such as thresholding, edge detection, region-based methods, or advanced deep learning approaches. By enabling precise localization and extraction of objects or regions, segmentation greatly enhances the capabilities of computer vision systems in understanding visual content and supporting a wide range of applications, including autonomous driving, medical imaging, and image editing.

link:

- <https://encord.com/blog/image-segmentation-for-computer-vision-best-practice-guide/#:~:text=Image%20segmentation%20is%20a%20crucial,%2C%20medical%20imaging%2C%20and%20robotics.>
- <https://www.telusinternational.com/insights/ai-data/article/guide-to-image-segmentation>
- <https://365datascience.com/tutorials/python-tutorials/build-custom-segmentation-models/>

Object Detection:

Object detection is a fundamental task in computer vision that involves identifying and localizing objects within images or videos. It goes beyond simple classification by not only recognizing the

presence of objects but also providing their precise locations. Object detection algorithms use a combination of techniques, such as feature extraction, machine learning, and deep learning, to achieve accurate and efficient results. These algorithms can detect and classify multiple objects simultaneously, even in complex and cluttered scenes. Object detection has diverse applications, including autonomous driving, surveillance, robotics, and augmented reality. By enabling machines to perceive and understand their visual environment, object detection plays a crucial role in advancing various fields and improving human-computer interaction.

link:

- https://drive.google.com/file/d/19sUoExTuCj9JhTO-sNLrATxzHal2YzRa/view?usp=drive_link
- https://drive.google.com/file/d/1Ap7rIUH7RM1Viv5kD1sSizjyQgLNh85/view?usp=drive_link
- <https://www.fritz.ai/object-detection/>
- <https://encord.com/blog/object-detection/>
- <https://www.javatpoint.com/object-recognition-using-python>

Project Practice:

[Object Detection using OpenCV \(Inference\) | Kaggle](#)

RESOURCES:

REFERENCE BOOKS:

- (CB) Pattern Recognition and Machine Learning, Christopher M. Bishop
- (KM) Machine Learning: a Probabilistic Perspective, Kevin Murphy
- (TM) Machine Learning, Tom Mitchell
- (HTF) The Elements of Statistical Learning: Data mining, Inference, and Prediction, by Hastie, Tibshirani, Friedman
- (DM) Information Theory, Inference, and Learning Algorithms, David Mackay
- Dive into Deep Learning: <https://d2l.ai/>

REFERENCE VIDEO:

- <https://drive.google.com/drive/folders/19owAy7ZBIHjk0ViHBfNOMJDZHgeXzXzK?usp=sharing> (AI MATERIAL)

- https://drive.google.com/drive/folders/1hQBM6wO6mPGYSQQf_98KzNacXBEI-maF?usp=sharing (FAST API)